IRON HACK

NATURAL LANGUAGE PROCESSING

HANDLING TEXT IN ML

- So far, the algorithms that we have seen take as input tabular information with numeric or categorical data.
- There are several areas of application where our data is not natively structured in this way: sound, image and text come to mind
- In these situations we often reduce the problem to turning the native data format into a suitable set of features, followed by applying the typical ML algorithms.
- In other words: how do we turn text into a suitable set of numbers?

HANDLING TEXT IN ML

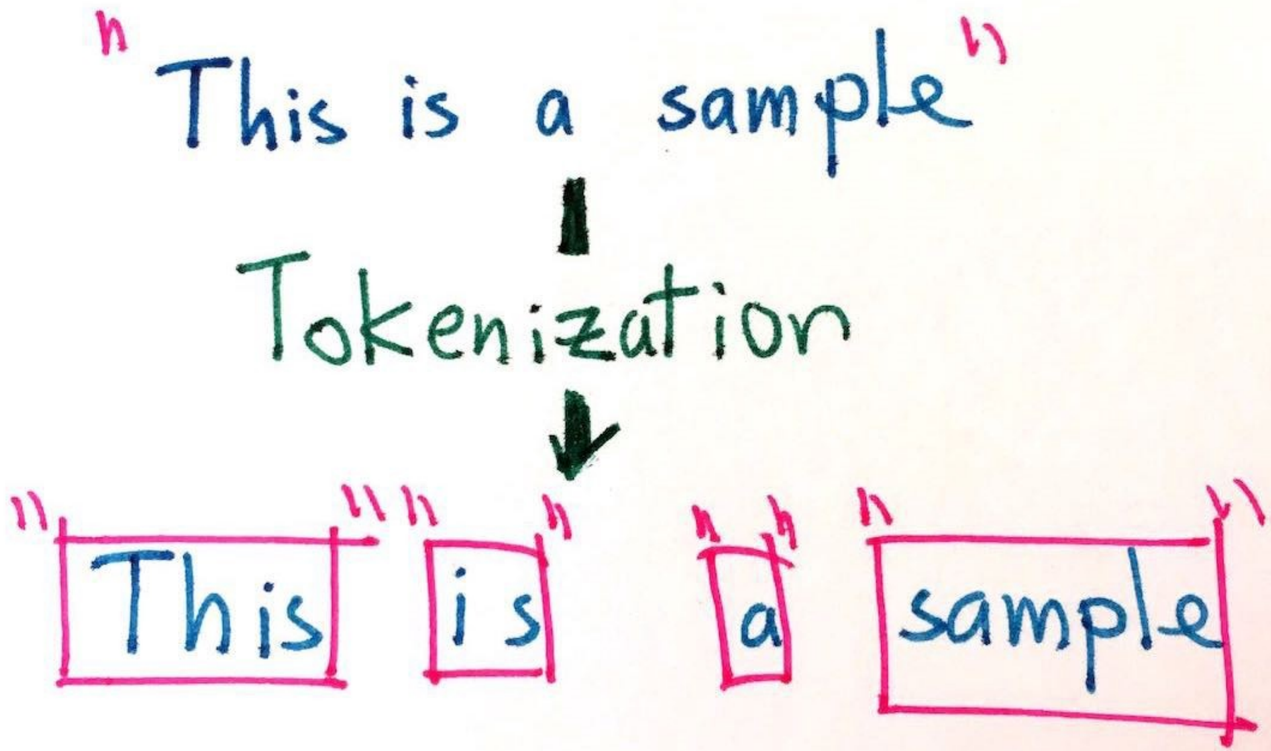
Some of this feature generation can and should come from metadata:

- source of text
- date of production
- author

But ultimately it is clear that most of the information must come from the body of the text.

TEXT DATA CLEANING TECHNIQUES

- Tokenization
- The process involves taking any text an splitting it into "tokens"
- Usually the tokens chosen are individua words but can be sentences as well.

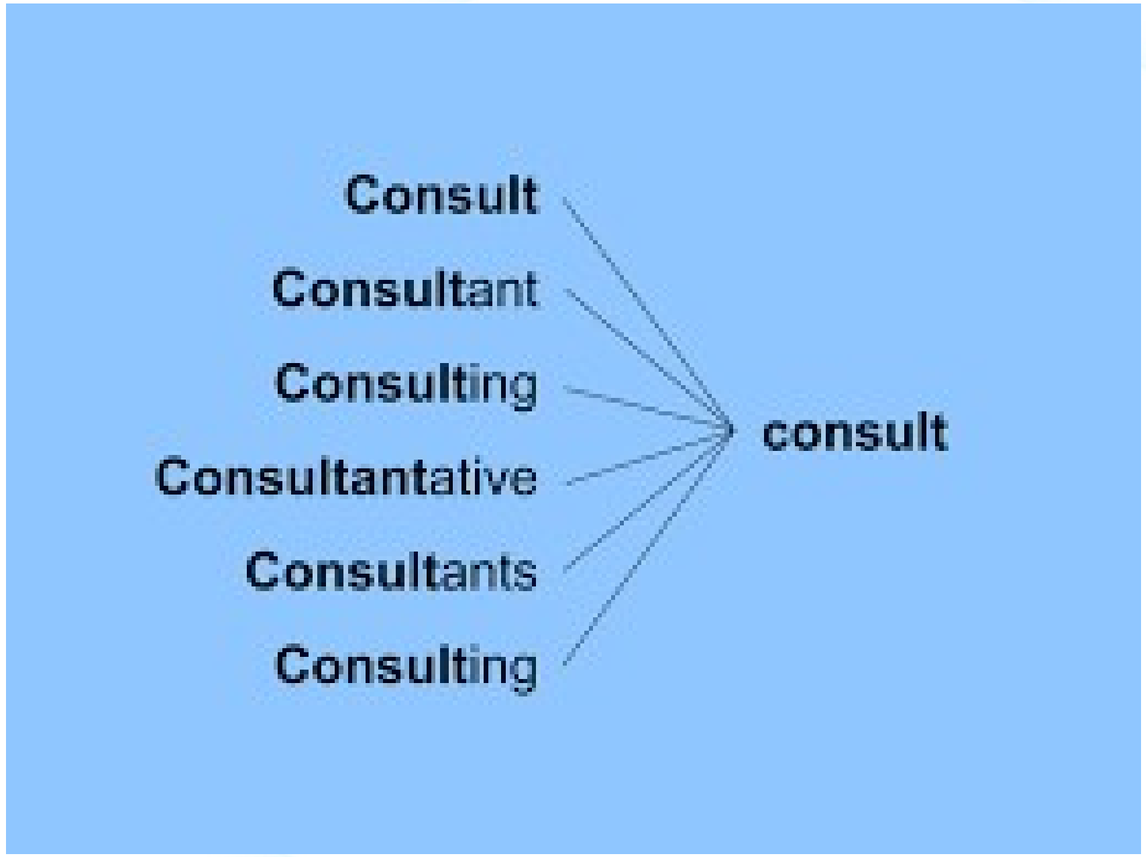


PROBLEMS: YOU CAN LOSE IMPORTANT MEANING LIKE "NEW YORK" GETS SPLIT INTO "NEW" AND "YORK" THUS LOSING THE GEOGRAPHICAL MEANING. A GOOD MACHINE LEARNING APPLICATION WOULD TAKE THIS INTO CONSIDERATION BY CONSIDERING THE RELATIVE FREQUENCY OF PAIRS OF WORDS AND PERHAPS KEEP THOSE PAIRS AS A SINGLE WORD

Jupyter

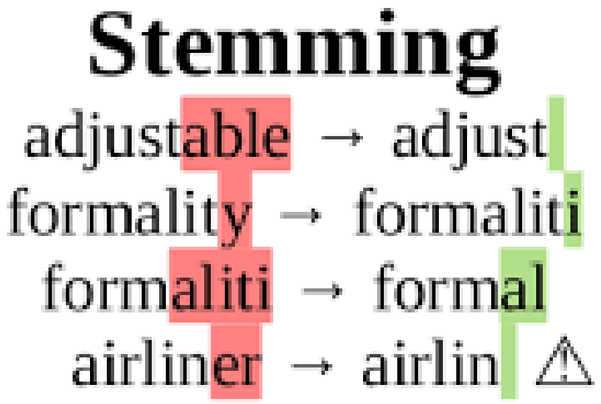
TEXT DATA CLEANING TECHNIQUES

- Stemming
- Removing prefixes and suffixes in order to reduce synonym words into their common form
- This can lead to some problems of creating spurious relations
- Stripes -> Strip
- Strip -> Strip

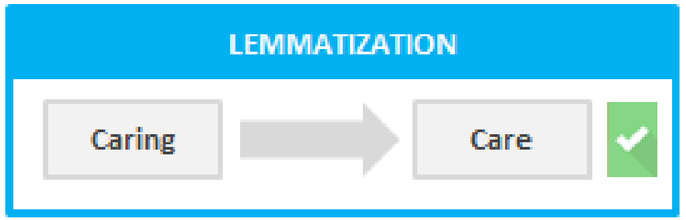


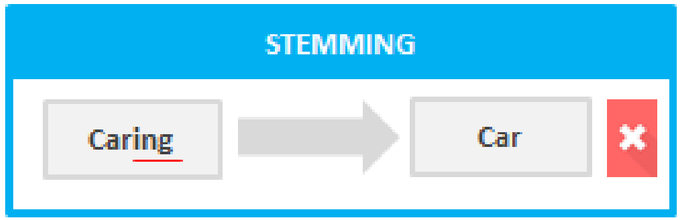
TEXT DATA CLEANING TECHNIQUES

lemmatization looks beyond word reduction and considers a language's full vocabulary to apply a morphological analysis to words. The lemma of 'was' is 'be' and the lemma of 'mice' is 'mouse'.









Jupyter

TEXT DATA CLEANING TECHNIQUES

- Part-of-Speech tagging (POS)
- Can be a very valuable feature
- Used heavily in Natural Language Generation (NLG)
- In that use case the "next word" is heavily dependent on the previous

POS tagging (Part-of-Speech tagging) in **Natural Language Processing (NLP)** is the process of **labeling each word in a sentence with its corresponding part of speech**, such as noun, verb, adjective, etc.

Jupyter

TEXT DATA CLEANING TECHNIQUES

Removal of stop words

Stop words are common words which are usually removed to not be interpreted as noise

A	It	These
About	Its	They
Again	Itself	This
All	Just	Those
Almost	km	Thus
Also	Made	To
Although	Mainly	Upon
Always	Make	Use
An	May	Used
And	mg	Using
Another	Might	Various
Any	ml	Very
Are	mm	Was
As	Most	We
At	Mostly	Were

To collab...

Jupyter

BAG OF WORDS

Bag of Words model is a standard way to decompose any text into several features that can then be used to train a classifier.

The method consists of taking all the words from all the corpuses/documents/rows and performing one-hot encoding. However, the final value is a frequency of the word rather than a binary variable
In essence, each distinct word becomes a feature



BAG OF WORDS


MARY
IS
HUNGRY
HAPPY
FOR
APPLES
NOT
JOHN
HE
"Mary is hungry for apples."
1 1 1 0 1 1 0 0 0
[1, 1, 1, 0, 1, 1, 0, 0, 0]
"John is happy he is not hungry for apples."
0 2 1 1 1 1 1 1 1
[0, 2, 1, 1, 1, 1, 1, 1, 1]

BAG OF WORDS

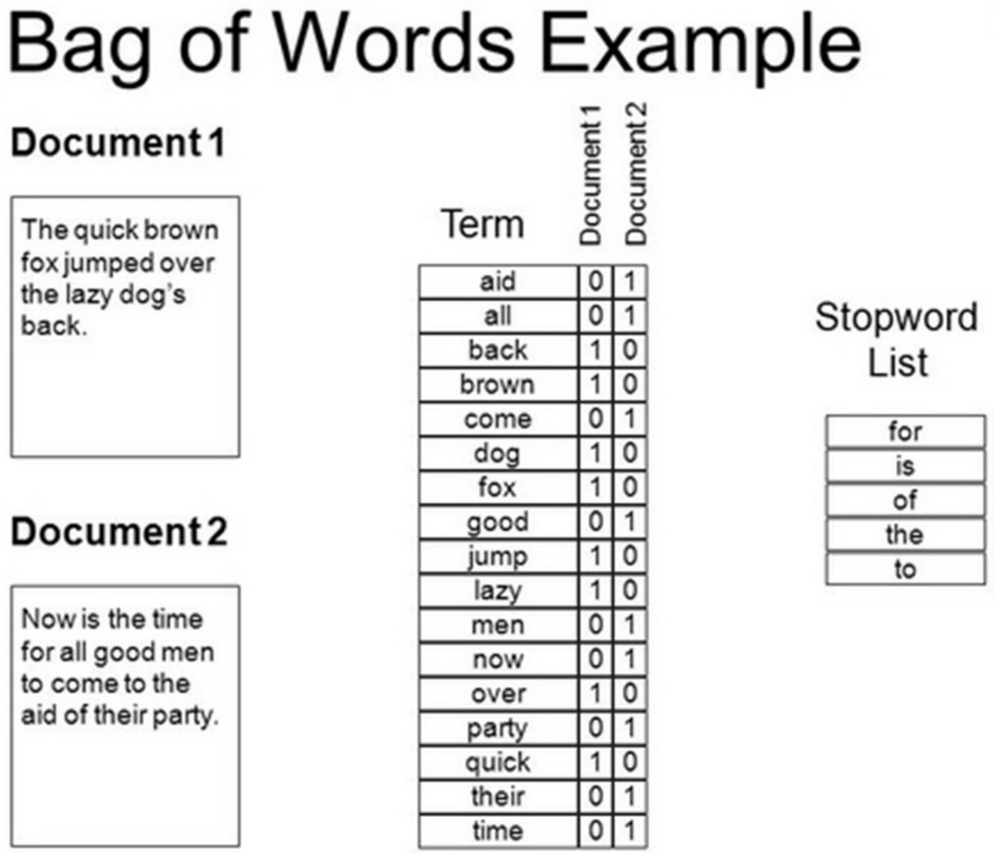
Problems of bag of words:

- you lose the context of the words within the sentence
- assumes all words are independent

"this is not bad" -> "bad"

solution: N-Grams performing bag of words but with pairs of sequential words for example:

"this is" ; "is not" ; "not bad" would be your features



Jupyter

TF-IDF

Term Frequency Inverse Document Frequency (TF-IDF) is one of the most important feature engineering techniques in natural language processing

It combines **two metrics**:

1. **Term Frequency (TF)** – How often a word appears in a document.

2. **Inverse Document Frequency (IDF)** – How unique or rare the word is across all documents.

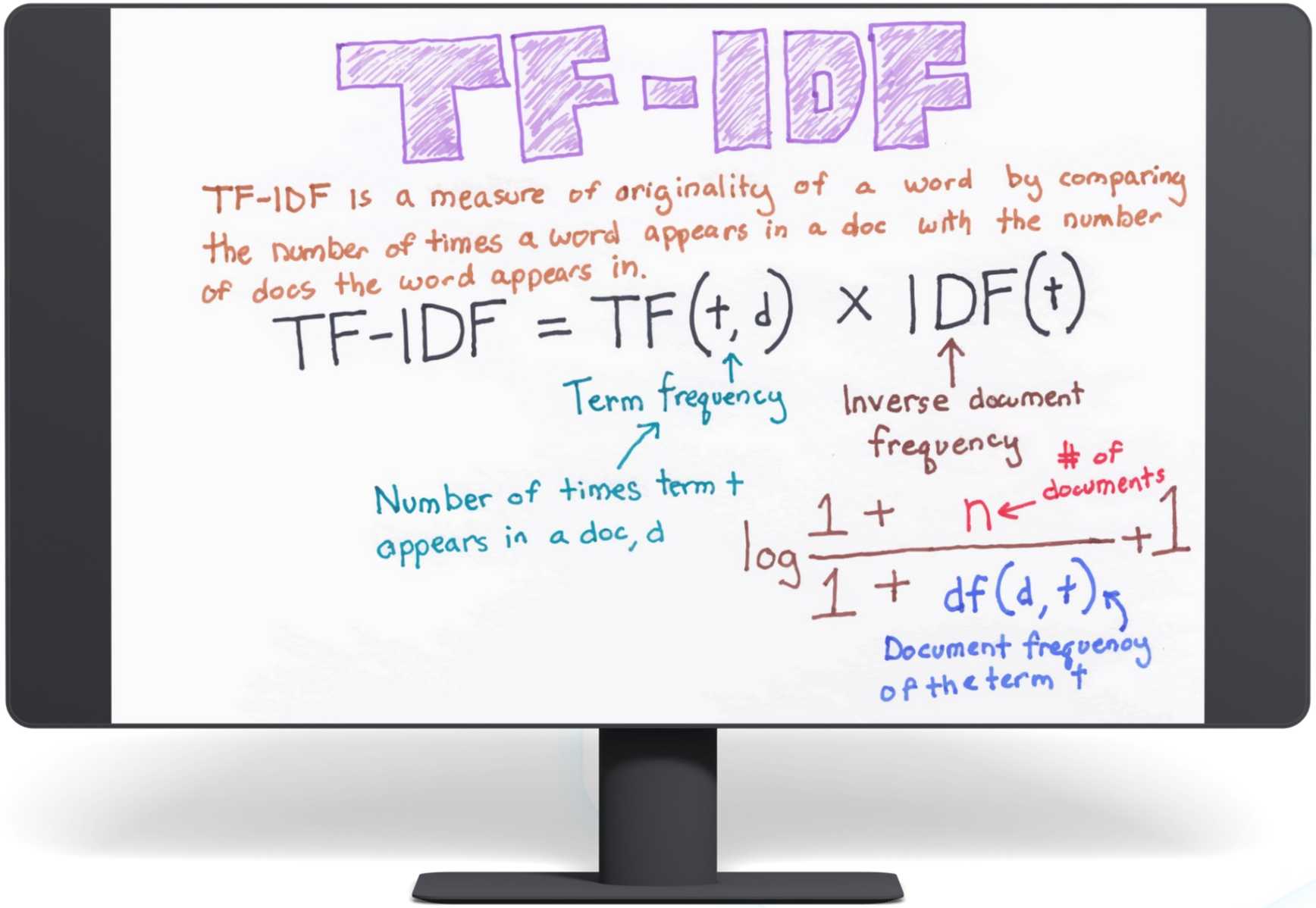
Weighs how important a word is in a specific document and how rare it is in all the collections of documents

TF-IDF

High TF-IDF score -> word appears many times in a document but is rare in collection of documents

Low TF-IDF score -> word that appears moderately in a document but is extremely common in collection of documents

TF-IDF


TF-IDF
TF-IDF is a measure of originality of a word by comparing the number of times a word appears in a doc with the number of docs the word appears in.
TF-IDF = TF(t, d) × IDF(t)
Term frequency
Number of times term t appears in a doc, d
Inverse document frequency
of documents
log (1 + n) / (1 + df(d, t)) + 1
Document frequency of the term t

ANY QUESTIONS ?

